Kubernetes Ingress 503 Service Unavailable Runbook

Document ID: RB-K8S-007 | Version: 1.0 | Last Updated: 2024-11-15

Category	kubernetes	**Severity**	P2 - High
Domain	Container Orchestration	**Est. Duration**	15-30 minutes
Tags	kubernetes, ingress, nginx, 503, service, endpoints, loadbalancer		

Overview

A 503 Service Unavailable from an Ingress controller means the backend Service has no healthy endpoints. This is distinct from a 502 (bad gateway — pods are up but returning errors) or a 404 (no routing rule). Common causes: all pods down, Service selector mismatch, readiness probe failing, or wrong port mapping.

Prerequisites

• kubectl configured for the cluster

• Access to Ingress controller logs (nginx-ingress or similar)

• curl or browser access to the failing endpoint

Response Steps

Step 1: Confirm 503 and Check Ingress Rule

Verify the error and check the Ingress resource is correctly configured with the right host and path.

```
curl -v https:///
kubectl get ingress -n
kubectl describe ingress -n
```

Step 2: Check Backend Service and Endpoints

Inspect the Service the Ingress points to and verify it has at least one healthy endpoint.

```
kubectl get service -n
kubectl get endpoints -n
kubectl describe endpoints -n
```

NOTE: Empty endpoints (no IP addresses listed) means no pods match the Service selector.

Step 3: Check Pod Selector Match

Verify the Service's label selector matches the labels on the pods.

```
kubectl get service -n -o jsonpath='{.spec.selector}'
kubectl get pods -n --show-labels
kubectl get pods -n -l =
```

Step 4: Check Readiness Probe

If pods are running but not in endpoints, the readiness probe is failing. Inspect and fix it.

```
kubectl describe pod -n | grep -A10 'Readiness'
kubectl logs -n | tail -50
kubectl get pod -n -o jsonpath='{.status.conditions}'
```

NOTE: A pod is removed from endpoints when its readiness probe fails, even if it is Running.

Step 5: Check Ingress Controller Logs

Inspect nginx-ingress or traefik logs for the specific error being returned.

```
kubectl get pods -n ingress-nginx
kubectl logs -n ingress-nginx | grep | tail -30
kubectl logs -n ingress-nginx --previous | tail -30
```

Step 6: Restart Pods and Verify

Restart the backend pods to clear transient readiness failures and verify endpoints populate.

```
kubectl rollout restart deployment/ -n
kubectl get endpoints -n -w
curl -v https:///
```

NOTE: SUCCESS: Endpoints show pod IPs and the URL returns 200 OK.

Rollback Steps

Rollback Step 1: Roll Back Deployment

If a recent deployment caused the 503, roll back.

```
kubectl rollout undo deployment/ -n
kubectl get endpoints -n
```

Rollback Step 2: Patch Readiness Probe

If the readiness probe is too strict, patch it with a longer timeout.

```
kubectl patch deployment -n --type='json' -p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/timeoutSeconds","value":10}]'
```

Step Dependencies

Step	Depends On	Can Parallelize With
Step 1: Confirm 503 + Ingress	—	—
Step 2: Check Service + Endpoints	Step 1	—
Step 3: Check Pod Selector	Step 2	Step 4
Step 4: Check Readiness Probe	Step 2	Step 3
Step 5: Check Ingress Logs	Step 1	Step 2, Step 3, Step 4
Step 6: Restart + Verify	Step 3, Step 4, Step 5	—
